.NET Ekosistemi

SDK • Visual Studio • VS Code • CLI

.NET 8 LTS	Cross-Platform	C# / F# / VB	ASP.NET Core
Entity Framework	Blazor	.NET MAUI	NuGet

MÜNDƏRİCAT

1. .NET Nədir?

- 1.1 .NET-in qısa tarixi
- 1.2 .NET Framework, Core, .NET 5+ — fərqlər
- 1.3 .NET 8-in yenilikləri

2. .NET SDK — Proqramçı Alət Dəsti

- 2.1 SDK nəyi əhatə edir?
- 2.2 Quraşdırma (Windows / macOS / Linux)
- 2.3 SDK, Runtime, Hosting Bundle fərqi

3. .NET CLI — Terminal Əmrləri

- 3.1 CLI nədir?
- 3.2 Ən çox işlədilən əmrlər
- 3.3 Layihə şablonları
- 3.4 NuGet paket idarəsi
- 3.5 Global alətlər

4. Visual Studio

- 4.1 Visual Studio nədir?
- 4.2 Community / Pro / Enterprise müqayisəsi
- 4.3 İnterfeysə baxış
- 4.4 Debug etmək **5.**

Visual Studio Code

- 5.1 VS Code nədir?
- 5.2 .NET üçün lazımlı əlavələr
- 5.3 Debug faylları: launch.json, tasks.json

6. Layihə Quruluşu

- 6.1 Qovluq strukturu
- 6.2 .csproj faylı nədir?
- 6.3 Program.cs — başlanğıc nöqtəsi

7. Hansı Növ Tətbiq Yaratmaq Olar?

- 7.1 Console, Web API, MVC, Blazor
- 7.2 .NET MAUI, Worker Service, gRPC

8. Hazır Məhsulu Paylaşmaq: Publish, Docker, CI/CD

- 8.1 dotnet publish
- 8.2 Docker ilə paketləmə
- 8.3 GitHub Actions ilə avtomatik deploy

.NET — Microsoft tərəfindən hazırlanmış, tamamilə pulsuz və açıq mənbəli proqram yazma platformasıdır. Bir dəfə öyrənib, üç əməliyyat sistemində (Windows, macOS, Linux) işləyən proqramlar yarada bilərsiniz. C#, F# və Visual Basic dillərini dəstəkləyir.

Sadə dillə desək: .NET, C# ilə proqram yazanda ehtiyac duyduğunuz hər şeyi — kompilyatoru, kitabxanaları, çalışdırma mühitini — bir yerdə toplayan çərçivədir.

1.1 Qısa Tarix — Haradan Hara Gəldik?

İl	Versiya	Nə dəyişdi?
2002	.NET Framework 1.0	İlk versiya — yalnız Windows
2016	.NET Core 1.0	Böyük dəyişiklik: Linux/macOS dəstəyi, açıq mənbə
2020	.NET 5	Framework + Core birləşdi, tək platforma oldu
2021	.NET 6 (LTS)	Minimal API, Hot Reload — daha sürətli inkişaf
2022	.NET 7	Performans yaxşılaşdırmalar, gRPC dəstəyi
2023	.NET 8 (LTS)	Native AOT, Blazor SSR, MAUI yeniləndi
2024	.NET 9	AI/ML inteqrasiyası, daha sürətli HTTP/3

1.2 .NET Framework vs .NET (müasir)

Köhnə .NET Framework (2002–2019) yalnız Windows-da işləyirdi. Müasir .NET isə (2016-dan bəri) hər platformada çalışır, daha yüngüldür və aktiv inkişaf etməyə davam edir. Yeni layihə başlayırsınızsa, həmişə müasir .NET-i seçin.

.NET Framework	.NET (müasir)
Yalnız Windows	Windows, macOS, Linux
Ağır quraşdırma	Yüngül, modul quruluş
GAC sistem qeydiyyatı	NuGet paket sistemi
Köhnə, dəstəklənmir	Aktiv inkişaf davam edir
Son versiya: 4.8.1	Cari versiya: .NET 8/9

1.3 .NET 8-in Ən Vacib Yenilikləri

- Native AOT — Proqram, kiçik ölçülü ikili fayla çevrilir. Başlama vaxtı çox azalır. Serverless / mikroservis üçün idealdır.
- Blazor SSR (Full Stack Web) — Eyni C# kodu həm serverdə, həm brauzerdə işləyir.
- C# 12 — Primary Constructor — Sinif yaratmaq daha qısa oldu.
- Frozen Collections — Dəyişilməz siyahılar. Paralel proqramlaşdırmada daha təhlükəsiz.
- Keyed DI Services — Eyni interfeysi paylaşan xidmətlər üçün açar əsaslı seçim.
- Performans — JSON emalı 20%, HTTP 15% daha sürətli (öncəki versiyaya nisbətən).

SDK (Software Development Kit) — proqram yazmaq, qurmaq, sınamaq və paylaşmaq üçün lazım olan bütün alətlər paketdir. Sadəcə SDK quraşdırırsınız — qalan hər şey (kompilyator, build sistemi, kitabxanalar) artıq içindədir.

Məşhur müqayisə: SDK, bir avtomobil sex dəstidir. Mühərriki (Runtime), alətləri (CLI), ehtiyat hissələri (kitabxanalar) bir qutuda gəlir.

2.1 SDK Nəyi Əhatə Edir?

Komponent	Nə işə yarayır?
dotnet CLI	Terminal əmrləri: layihə yaratmaq, qurmaq, işlətmək
Roslyn	C# / VB.NET kodunu maşın koduna çevirən kompilyator
MSBuild	Build sistemi — .csproj fayllarını oxuyur, işlədir
.NET Runtime	Yazılan proqramı icra edən virtual mühit (CLR)
BCL	Hazır kitabxanalar: String, List, DateTime, LINQ, IO...
ASP.NET Core	Veb tətbiq, API, real-time üçün çərçivə
NuGet Client	Xarici paketlər yükləmək / idarə etmək

Windows (winget ilə)

```
$ winget install Microsoft.DotNet.SDK.8
```

macOS (Homebrew ilə)

```
$ brew install --cask dotnet-sdk
```

Ubuntu / Debian

```
$ sudo apt-get install -y dotnet-sdk-8.0
```

Fedora / RHEL

```
$ sudo dnf install dotnet-sdk-8.0
```

Quraşdırdıqdan sonra terminala bu əmri yazıb yoxlayın:

```
$ dotnet --version # məsələn: 8.0.300
```

```
$ dotnet --list-sdks # quraşdırılmış bütün SDK-lar
```

2.3 SDK, Runtime və Hosting Bundle — Fərq Nədir?

Paket	Nə üçündür?
Runtime	Hazır proqramı işlətmək üçün minimal paket. Server mühiti üçün kifayətdir.
SDK	Runtime + kompilyator + CLI + build alətləri. İnkişaf üçün lazımdır.
Hosting Bundle	Windows serverlərində IIS-lə ASP.NET Core tətbiqi qoymaq üçün.

⚠ **Vacib qeyd**

Serverdə yalnız Runtime quraşdırın — SDK-ya məhsuldar mühitdə ehtiyac yoxdur. Bu, həm təhlükəsizlik, həm də disk yeri baxımından düzgün yanaşmadır.

CLI (Command Line Interface) — Visual Studio olmadan, sadəcə terminal istifadə edərək .NET layihəsi yaratmaq, qurmaq, sınamaq və paylaşmaq imkanı verir. CI/CD pipeline-larında, Linux serverlərində və yüngül iş axınlarında CLI-dan istifadə çox yaygındır.

Bütün CLI əmrləri `dotnet` sözü ilə başlayır, məsələn: `dotnet build`

3.1 Ən Çox İşlədilən Əmrlər

Əmr	Nə edir?
`dotnet new <şablon>`	Yeni layihə yaradır (console, webapi, mvc, ...)
`dotnet restore`	NuGet paketlərini yükləyir / bərpa edir
`dotnet build`	Layihəni kompilyasiya edir, səhvləri göstərir
`dotnet run`	Build + icra — birlikdə
`dotnet watch`	Kodu dəyişdikdə avtomatik yenidən başlayır
`dotnet test`	Unit testlərini işlədir, nəticə göstərir
`dotnet publish`	Dağıtım üçün hazır çıxış qovluğu yaradır
`dotnet add package`	NuGet paketi əlavə edir
`dotnet remove package`	NuGet paketini çıxarır
`dotnet list package`	Cari paketləri göstərir
`dotnet tool install`	Global CLI aləti quraşdırır
`dotnet format`	Kodu standart formata salır
`dotnet sln`	Solution (.sln) faylını idarə edir
`dotnet ef`	Entity Framework Core miqrasiyaları

3.2 Layihə Şablonları

```
$ dotnet new list # mövcud bütün şablonları gör
$ dotnet new console -n MyApp # sadə terminal proqramı
$ dotnet new webapi -n MyApi # REST API
$ dotnet new mvc -n MyWeb # MVC veb tətbiqi
$ dotnet new blazorwasm -n MyBlazor # Blazor WebAssembly
$ dotnet new classlib -n MyLibrary # sinif kitabxanası
$ dotnet new xunit -n MyTests # xUnit test layihəsi
$ dotnet new worker -n MyWorker # arxa plan xidməti
```

```
$ dotnet new maui -n MyMauiApp # mobil / masaüstü tətbiq
```

3.3 NuGet Paket İdarəsi

```
# Paket əlavə et
$ dotnet add package Newtonsoft.Json
$ dotnet add package Microsoft.EntityFrameworkCore --version 8.0.0

# Mövcud paketlər
$ dotnet list package
$ dotnet list package --outdated # yenilənməyə ehtiyacı olanlar

# Paket sil
$ dotnet remove package Newtonsoft.Json

# Paketləri bərpa et (git clone-dan sonra)
$ dotnet restore
```

dotnet watch — Ən çox sevdiyiniz əmr olacaq!

Terminal açıq saxlayıb `dotnet watch run` yazın. Artıq hər fayl saxladıqda tətbiq özü-özünü yenidən başladır. Brauzeri yeniləməyə belə ehtiyac qalmır!

Visual Studio (VS) — Microsoft-un hazırladığı, .NET üçün ən zəngin xüsusiyyətlərə malik tam IDE-dir (Integrated Development Environment — Bütünləşmiş İnkişaf Mühiti). Windows-da ən populyar .NET inkişaf alətidir. macOS versiyası da mövcuddur, lakin Windows versiyası daha güclüdür.

4.1 Hansı Versiyası Var?

Versiya	Qiymət	Kim üçün?
Community	Pulsuz	Şəxsi, açıq mənbə, kiçik şirkət
Professional	~$499/il	Kommersiya layihələr üçün
Enterprise	~$2999/il	Böyük komandalar, xüsusi analiz alətləri

Əgər tələbəsinizsə, fərdi layihə inkişaf etdirirsinizsə və ya açıq mənbə layihəsindən işləyirsinizsə — Community versiyası tamamilə kifayətdir.

4.2 Əsas İnterfeys Panelləri

- Solution Explorer — Bütün layihə fayllarını ağac şəklində göstərir. Sol tərəfdə olur.
- Kod Redaktoru — Sözdizimi rəngləmə, IntelliSense (avtotamamlama), refactoring.
- Output Window — Build nəticəsi, xəta mesajları, log çıxışı.
- Error List — Bütün xəta və xəbərdarlıqların siyahısı. İkliklə koda keçir.
- NuGet Package Manager — Paketlər axtarıb quraşdırmaq üçün vizual interfeys.

- Test Explorer — Bütün unit testlər. Bir klikdə hamısını işlət.

- Git Changes — Dəyişiklikləri gör, commit et, push et — Visual Studio-dan çıxmadan.
- IntelliCode — AI dəstəkli kod tamamlama — ən çox istifadə edilən metodları üstdə göstərir.

4.3 Debug Etmək — Addım-addım

Debug — proqramın niyə yanlış işlədiyini anlamaq üçün addım-addım izləmə prosesidir.

Visual Studio bu sahədə ən güclü alətlərdən birini təqdim edir:

- F9 — Breakpoint qoy (proqram həmin sətirdə dayanacaq)
- F5 — Debug rejimini başlat
- F10 — Növbəti sətirə keç (funksiyaya girmir)
- F11 — Funksiya içinə gir
- Shift+F5 — Debug-u dayandır
- Watch Window — Dəyişənin dəyərini real vaxtda izlə
- Conditional Breakpoint — Yalnız şərt ödənildikdə dayansın (sağ klik → şərt əlavə et)
- Hot Reload — Debug zamanı kodu dəyiş, proqram dayanmadan yenilənir

VS Code — yüngül, sürətli, tamamilə pulsuz kod redaktorudur. Microsoft tərəfindən hazırlanıb, hər platformada işləyir. Visual Studio-dan fərqli olaraq çox az resurs istehlak edir. C# Dev Kit əlavəsi quraşdırıldıqdan sonra Visual Studio-ya yaxın imkanlara malik olur.

Fərq nədir? Visual Studio — tam hazır IDE (ağır amma güclü). VS Code — yüngül redaktor + istədiyiniz əlavələr (siz konfiqurasiya edirsiniz).

5.1 .NET üçün Lazımlı Əlavələr

Əlavə	Hazırlayan	Nə işə yarayır?
C# Dev Kit	Microsoft	IntelliSense, debug, test — əsas əlavə
.NET Install Tool	Microsoft	Lazımlı SDK-nı avtomatik yükləyir
NuGet Gallery	pcislo	GUI ilə paket axtarışı
REST Client	Huachao	.http faylları ilə API sınağı — Postman lazım deyil
GitLens	GitKraken	Git tarixçəsini kod sətirləri üzərindən göstərir
Docker	Microsoft	Dockerfile, docker-compose dəstəyi
Error Lens	Alexander	Xətaları birbaşa kod sətirinin yanında göstərir
Roslynator	Josef Pihrt	500+ C# analiz və refactoring qaydası

5.2 launch.json — Debug Konfiqurasiyası

Bu fayl VS Code-a layihənizi necə debug edəcəyini izah edir. `.vscode/launch.json` yolunda saxlanılır:

```
{
"version": "0.2.0",
"configurations": [
{
"name": "Layihəni Debug et",
"type": "coreclr",
"request": "launch",
"preLaunchTask": "build",
"program": "${workspaceFolder}/bin/Debug/net8.0/MyApp.dll",
"env": { "ASPNETCORE_ENVIRONMENT": "Development" }
}
]
}
```

5.3 tasks.json — Avtomatik Build

Bu fayl VS Code-a F5 basıldığında əvvəlcə build etməyi bildirir:

```
{
"version": "2.0.0",
"tasks": [
{
"label": "build",
"command": "dotnet",
"type": "process",
"args": ["build", "${workspaceFolder}", "-c", "Debug"],
"group": { "kind": "build", "isDefault": true },
"problemMatcher": "$msCompile"
}
]
}
```

6.1 Tipik Qovluq Strukturu (Web API)

```
MyApi/
├── MyApi.sln # Bütün layihələri birləşdirən fay
├── src/
│ └── MyApi/
│ ├── MyApi.csproj # Layihə faylı (paketlər, versiya...)
│ ├── Program.cs # Proqramın başladığı yer
│ ├── appsettings.json # Konfiqurasiya (DB, API key...)
│ ├── Controllers/ # API son nöqtələri (endpoint-lər)
│ ├── Models/ # Məlumat modelləri (siniflər)
│ ├── Services/ # İş məntiqi
│ └── Repositories/ # Verilənlər bazasına müraciət
└── tests/
└── MyApi.Tests/
└── Controllers/ # Unit testlər
```

6.2 .csproj Faylı — Layihənin Pasportu

.csproj faylı layihənizin əsas məlumatlarını saxlayır: hansı .NET versiyasını istifadə etdiyini, hansı paketlərə ehtiyac olduğunu, hansı xüsusiyyətlərin aktiv olduğunu. Visual Studio / CLI bu faylı oxuyaraq işləyir.

```
net8.0

enable

enable
```

6.3 Program.cs — Proqramın Başladığı Yer

```
// 1) Builder: xidmətləri qeyd et var builder =
WebApplication.CreateBuilder(args);

builder.Services.AddControllers(); // Controller-ları əlavə et
builder.Services.AddEndpointsApiExplorer(); // Swagger üçün
builder.Services.AddSwaggerGen(); // API sənədləşməsi
builder.Services.AddDbContext(opt =>
opt.UseSqlServer(builder.Configuration
.GetConnectionString("Default")));

// 2) Tətbiq: middleware pipeline qur
var app = builder.Build();

if (app.Environment.IsDevelopment()) {
app.UseSwagger(); // Swagger aktiv (yalnız
inkişafda) app.UseSwaggerUI(); // Swagger UI }

app.UseHttpsRedirection(); // HTTP-ni HTTPS-ə yönləndir
app.UseAuthorization(); // Səlahiyyət yoxlaması
app.MapControllers(); // Controller route-larını qeyd et

app.Run(); // Serveri başlat
```

Tətbiq Növü	Şablon	İstifadə Yeri
Console App	`dotnet new console`	Terminal proqramı. Skriptlər, alətlər.
Web API	`dotnet new webapi`	REST API. Mobil app backend, mikroservis.
MVC Web App	`dotnet new mvc`	Server-side HTML. Ənənəvi veb sayt.
Razor Pages	`dotnet new webapp`	Sadə veb tətbiq. MVC-dən az kod.
Blazor Server	`dotnet new blazorserver`	Serverdə işləyən interaktiv UI.
Blazor WebAssembly	`dotnet new blazorwasm`	C# birbaşa brauzerdə işləyir!
.NET MAUI	`dotnet new maui`	iOS, Android, Windows, macOS — bir kod.
Worker Service	`dotnet new worker`	Arxa plan tapşırığı, scheduled job.
gRPC Service	`dotnet new grpc`	Sürətli binary protokol. Mikroservislər.
Class Library	`dotnet new classlib`	Paylaşılan kod. NuGet paketi ola bilər.

Hansını seçim?

Yeni başlayırsınızsa — Console App ilə başlayın. Veb API qurmaq istəyirsinizsə — Web API şablonunu seçin. Bir kod ilə telefon + masaüstü tətbiqi istəyirsinizsə — .NET MAUI-yə baxın.

8. Hazır Məhsulu Paylaşmaq

Proqramı hazır etdikdən sonra `dotnet publish` əmri onu serverə və ya bulud mühitinə yerləşdirmək üçün hazırlayır:

```
# Ən sadə variant (Framework-dependent — serverda Runtime olmalıdır)
$ dotnet publish -c Release -o ./publish

# Self-contained (Runtime daxil, server lazımsız)
$ dotnet publish -c Release -r linux-x64 --self-contained true -o ./publish

# Native AOT (ən kiçik, ən sürətli başlanğıc)
$ dotnet publish -c Release -r linux-x64 /p:PublishAot=true -o ./publish

# Tək fayl (bütün şeylər bir .exe-də)
$ dotnet publish -c Release -r win-x64 /p:PublishSingleFile=true -o ./publish
```

8.2 Docker ilə Konteynerləşdirmə

Docker, proqramınızı istənilən serverda eyni şəkildə işlətməyə imkan verir. Aşağıdakı

Dockerfile iki mərhələli build istifadə edir: ağır SDK mərhələsi yalnız build üçün, yüngül Runtime mərhələsi isə son imaj üçün:

```
# --- Build mərhələsi ---
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
WORKDIR /src
COPY *.csproj .
RUN dotnet restore
COPY . .
RUN dotnet publish -c Release -o /app/publish

# --- Runtime mərhələsi (kiçik final imaj) ---
FROM mcr.microsoft.com/dotnet/aspnet:8.0
WORKDIR /app COPY --from=build
/app/publish .
EXPOSE 8080
ENTRYPOINT ["dotnet", "MyApi.dll"]

# Docker əmrləri:
$ docker build -t my-api:latest .
$ docker run -d -p 8080:8080 my-api:latest
```

8.3 GitHub Actions — Avtomatik CI/CD

Kodu GitHub-a push etdikdə aşağıdakı iş axını avtomatik işə düşür: build, test, publish.

Hər commit-dən sonra əl ilə heç nə etmək lazım deyil:

```
name: .NET
CI/CD on: push:
branches: [ main ]

jobs: build-test-publish:
runs-on: ubuntu-latest
steps: - uses:
actions/checkout@v4

-      name: .NET quraşdıruses: actions/setup-dotnet@v4 with: dotnet-version: '8.0.x'

-      run: dotnet restore
-      run: dotnet build --no-restore -c Release
-      run: dotnet test --no-build -c Release
-      run: dotnet publish -c Release -o ./publish
```

Performans Məsləhəti

Production mühitlərində `PublishReadyToRun=true` parametrindən istifadə edin. Bu, proqramın başlanğıc vaxtını əhəmiyyətli dərəcədə azaldır.

Bu bələdçi Microsoft-un rəsmi sənədlərindən ilham alaraq Azərbaycan dilindəki inkişafçılar üçün hazırlanmışdır. Ətraflı məlumat: docs.microsoft.com/dotnet